Evaluations (often called **evals**) test model outputs to ensure they meet style and content criteria that you specify. Writing evals to understand how your LLM applications are performing against your expectations, especially when upgrading or trying new models, is an essential component to building reliable applications.

In this guide, we will focus on **configuring evals programmatically using the Evals API**. If you prefer, you can also configure evals in the OpenAI dashboard.

If you're new to evaluations, or want a more iterative environment to experiment in as you build your eval, consider trying Datasets instead.

Broadly, there are three steps to build and run evals for your LLM application.

1. Describe the task to be done as an eval
2. Run your eval with test inputs (a prompt and input data)
3. Analyze the results, then iterate and improve on your prompt

This process is somewhat similar to behavior-driven development (BDD), where you begin by specifying how the system should behave before implementing and testing the system. Let's see how we would complete each of the steps above using the Evals API.

Create an eval for a task

Creating an eval begins by describing a task to be done by a model. Let's say that we would like to use a model to classify the contents of IT support tickets into one of three categories: `Hardware`, `Software`, or `Other`.

To implement this use case, you can use either the Chat Completions API or the Responses API. Both examples below combine a developer message with a user message containing the text of a support ticket.

Categorize IT support tickets

```
curl https://api.openai.com/v1/responses \\
    -H "Authorization: Bearer $OPENAI_API_KEY" \\
    -H "Content-Type: application/json" \\
    -d '{
        "model": "gpt-4.1",
        "input": [
            {
                "role": "developer",
```

```
                "content": "Categorize the following support ticket into one
of Hardware, Software, or Other."
            },
            {
                "role": "user",
                "content": "My monitor wont turn on - help!"
            }
        ]
    }'
```

```
import OpenAI from "openai";
const client = new OpenAI();

const instructions = \`
You are an expert in categorizing IT support tickets. Given the support
ticket below, categorize the request into one of "Hardware", "Software",
or "Other". Respond with only one of those words.
\`;

const ticket = "My monitor won't turn on - help!";

const response = await client.responses.create({
    model: "gpt-4.1",
    input: [
        { role: "developer", content: instructions },
        { role: "user", content: ticket },
    ],
});

console.log(response.output_text);
```

```
from openai import OpenAI
client = OpenAI()

instructions = """
You are an expert in categorizing IT support tickets. Given the support
ticket below, categorize the request into one of "Hardware", "Software",
or "Other". Respond with only one of those words.
"""

ticket = "My monitor won't turn on - help!"

response = client.responses.create(
    model="gpt-4.1",
```

```
    input=[
        {"role": "developer", "content": instructions},
        {"role": "user", "content": ticket},
    ],
)

print(response.output_text)
```

Let's set up an eval to test this behavior via API. An eval needs two key ingredients:

- `data_source_config` : A schema for the test data you will use along with the eval.
- `testing_criteria` : The graders that determine if the model output is correct.

Create an eval

```
curl https://api.openai.com/v1/evals \\
    -H "Authorization: Bearer $OPENAI_API_KEY" \\
    -H "Content-Type: application/json" \\
    -d '{
        "name": "IT Ticket Categorization",
        "data_source_config": {
            "type": "custom",
            "item_schema": {
                "type": "object",
                "properties": {
                    "ticket_text": { "type": "string" },
                    "correct_label": { "type": "string" }
                },
                "required": ["ticket_text", "correct_label"]
            },
            "include_sample_schema": true
        },
        "testing_criteria": [
            {
                "type": "string_check",
                "name": "Match output to human label",
                "input": "{{ sample.output_text }}",
                "operation": "eq",
                "reference": "{{ item.correct_label }}"
            }
        ]
    }'
```

```
import OpenAI from "openai";
const openai = new OpenAI();
```

```
const evalObj = await openai.evals.create({
    name: "IT Ticket Categorization",
    data_source_config: {
        type: "custom",
        item_schema: {
            type: "object",
            properties: {
                ticket_text: { type: "string" },
                correct_label: { type: "string" }
            },
            required: ["ticket_text", "correct_label"],
        },
        include_sample_schema: true,
    },
    testing_criteria: [
        {
            type: "string_check",
            name: "Match output to human label",
            input: "{{ sample.output_text }}",
            operation: "eq",
            reference: "{{ item.correct_label }}",
        },
    ],
});

console.log(evalObj);
```

```
from openai import OpenAI
client = OpenAI()

eval_obj = client.evals.create(
    name="IT Ticket Categorization",
    data_source_config={
        "type": "custom",
        "item_schema": {
            "type": "object",
            "properties": {
                "ticket_text": {"type": "string"},
                "correct_label": {"type": "string"},
            },
            "required": ["ticket_text", "correct_label"],
        },
        "include_sample_schema": True,
    },
```

```
    testing_criteria=[
        {
            "type": "string_check",
            "name": "Match output to human label",
            "input": "{{ sample.output_text }}",
            "operation": "eq",
            "reference": "{{ item.correct_label }}",
        }
    ],
)

print(eval_obj)
```

Explanation: data_source_config parameter

Running this eval will require a test data set that represents the type of data you expect your prompt to work with (more on creating the test data set later in this guide). In our `data_source_config` parameter, we specify that each **item** in the data set will conform to a JSON schema with two properties:

- `ticket_text`: a string of text with the contents of a support ticket
- `correct_label`: a "ground truth" output that the model should match, provided by a human

Since we will be referencing a **sample** in our test criteria (the output generated by a model given our prompt), we also set `include_sample_schema` to `true`.

```
{
  "type": "custom",
  "item_schema": {
    "type": "object",
    "properties": {
      "ticket": { "type": "string" },
      "category": { "type": "string" }
    },
    "required": ["ticket", "category"]
  },
  "include_sample_schema": true
}
```

Explanation: testing_criteria parameter

In our `testing_criteria`, we define how we will conclude if the model output satisfies our requirements for each item in the data set. In this case, we just want the model to output one of three category strings based on the input ticket. The string it outputs should exactly match the

human-labeled `correct_label` field in our test data. So in this case, we will want to use a `string_check` grader to evaluate the output.

In the test configuration, we will introduce template syntax, represented by the `{{` and `}}` brackets below. This is how we will insert dynamic content into the test for this eval.

- `{{ item.correct_label }}` refers to the ground truth value in our test data.
- `{{ sample.output_text }}` refers to the content we will generate from a model to evaluate our prompt - we'll show how to do that when we actually kick off the eval run.

```
{
  "type": "string_check",
  "name": "Category string match",
  "input": "{{ sample.output_text }}",
  "operation": "eq",
  "reference": "{{ item.category }}"
}
```

After creating the eval, it will be assigned a UUID that you will need to address it later when kicking off a run.

```
{
  "object": "eval",
  "id": "eval_67e321d23b54819096e6bfe140161184",
  "data_source_config": {
    "type": "custom",
    "schema": { ... omitted for brevity... }
  },
  "testing_criteria": [
    {
      "name": "Match output to human label",
      "id": "Match output to human label-c4fdf789-2fa5-407f-8a41-
a6f4f9afd482",
      "type": "string_check",
      "input": "{{ sample.output_text }}",
      "reference": "{{ item.correct_label }}",
      "operation": "eq"
    }
  ],
  "name": "IT Ticket Categorization",
  "created_at": 1742938578,
  "metadata": {}
}
```

Now that we've created an eval that describes the desired behavior of our application, let's test a prompt with a set of test data.

Test a prompt with your eval

Now that we have defined how we want our app to behave in an eval, let's construct a prompt that reliably generates the correct output for a representative sample of test data.

Uploading test data

There are several ways to provide test data for eval runs, but it may be convenient to upload a JSONL file that contains data in the schema we specified when we created our eval. A sample JSONL file that conforms to the schema we set up is below:

```
{ "item": { "ticket_text": "My monitor won't turn on!", "correct_label":
"Hardware" } }
{ "item": { "ticket_text": "I'm in vim and I can't quit!", "correct_label":
"Software" } }
{ "item": { "ticket_text": "Best restaurants in Cleveland?", "correct_label":
"Other" } }
```

This data set contains both test inputs and ground truth labels to compare model outputs against.

Next, let's upload our test data file to the OpenAI platform so we can reference it later. You can upload files in the dashboard here, but it's possible to upload files via API as well. The samples below assume you are running the command in a directory where you saved the sample JSON data above to a file called `tickets.jsonl`:

Upload a test data file

```
curl https://api.openai.com/v1/files \\
  -H "Authorization: Bearer $OPENAI_API_KEY" \\
  -F purpose="evals" \\
  -F file="@tickets.jsonl"
```

```
import fs from "fs";
import OpenAI from "openai";

const openai = new OpenAI();

const file = await openai.files.create({
    file: fs.createReadStream("tickets.jsonl"),
```

```
    purpose: "evals",
});

console.log(file);
```

```
from openai import OpenAI
client = OpenAI()

file = client.files.create(
    file=open("tickets.jsonl", "rb"),
    purpose="evals"
)

print(file)
```

When you upload the file, make note of the unique `id` property in the response payload (also available in the UI if you uploaded via the browser) - we will need to reference that value later:

```
{
  "object": "file",
  "id": "file-CwHg45Fo7YXwkWRPUkLNHW",
  "purpose": "evals",
  "filename": "tickets.jsonl",
  "bytes": 208,
  "created_at": 1742834798,
  "expires_at": null,
  "status": "processed",
  "status_details": null
}
```

Creating an eval run

With our test data in place, let's evaluate a prompt and see how it performs against our test criteria. Via API, we can do this by creating an eval run.

Make sure to replace `YOUR_EVAL_ID` and `YOUR_FILE_ID` with the unique IDs of the eval configuration and test data files you created in the steps above.

Create an eval run

```
curl https://api.openai.com/v1/evals/YOUR_EVAL_ID/runs \\
    -H "Authorization: Bearer $OPENAI_API_KEY" \\
    -H "Content-Type: application/json" \\
    -d '{
```

```
        "name": "Categorization text run",
        "data_source": {
            "type": "responses",
            "model": "gpt-4.1",
            "input_messages": {
                "type": "template",
                "template": [
                    {"role": "developer", "content": "You are an expert in
categorizing IT support tickets. Given the support ticket below, categorize
the request into one of Hardware, Software, or Other. Respond with only one of
those words."},
                    {"role": "user", "content": "{{ item.ticket_text }}"}
                ]
            },
            "source": { "type": "file_id", "id": "YOUR_FILE_ID" }
        }
    }'
```

```
import OpenAI from "openai";
const openai = new OpenAI();

const run = await openai.evals.runs.create("YOUR_EVAL_ID", {
    name: "Categorization text run",
    data_source: {
        type: "responses",
        model: "gpt-4.1",
        input_messages: {
            type: "template",
            template: [
                { role: "developer", content: "You are an expert in
categorizing IT support tickets. Given the support ticket below, categorize
the request into one of 'Hardware', 'Software', or 'Other'. Respond with only
one of those words." },
                { role: "user", content: "{{ item.ticket_text }}" },
            ],
        },
        source: { type: "file_id", id: "YOUR_FILE_ID" },
    },
});

console.log(run);
```

```
from openai import OpenAI
client = OpenAI()
```

```
run = client.evals.runs.create(
    "YOUR_EVAL_ID",
    name="Categorization text run",
    data_source={
        "type": "responses",
        "model": "gpt-4.1",
        "input_messages": {
            "type": "template",
            "template": [
                {"role": "developer", "content": "You are an expert in
categorizing IT support tickets. Given the support ticket below, categorize
the request into one of 'Hardware', 'Software', or 'Other'. Respond with only
one of those words."},
                {"role": "user", "content": "{{ item.ticket_text }}"},
            ],
        },
        "source": {"type": "file_id", "id": "YOUR_FILE_ID"},
    },
)

print(run)
```

When we create the run, we set up a prompt using either a Chat Completions messages array or a Responses input. This prompt is used to generate a model response for every line of test data in your data set. We can use the double curly brace syntax to template in the dynamic variable `item.ticket_text`, which is drawn from the current test data item.

If the eval run is successfully created, you'll receive an API response that looks like this:


```
{
    "object": "eval.run",
    "id": "evalrun_67e44c73eb6481909f79a457749222c7",
    "eval_id": "eval_67e44c5becec81909704be0318146157",
    "report_url": "https://platform.openai.com/evaluation/evals/abc123",
    "status": "queued",
    "model": "gpt-4.1",
    "name": "Categorization text run",
    "created_at": 1743015028,
    "result_counts": { ... },
    "per_model_usage": null,
    "per_testing_criteria_results": null,
    "data_source": {
        "type": "responses",
        "source": {
```

```
            "type": "file_id",
            "id": "file-J7MoX9ToHXp2TutMEeYnwj"
        },
        "input_messages": {
            "type": "template",
            "template": [
                {
                    "type": "message",
                    "role": "developer",
                    "content": {
                        "type": "input_text",
                        "text": "You are an expert in...."
                    }
                },
                {
                    "type": "message",
                    "role": "user",
                    "content": {
                        "type": "input_text",
                        "text": "{{item.ticket_text}}"
                    }
                }
            ]
        },
        "model": "gpt-4.1",
        "sampling_params": null
    },
    "error": null,
    "metadata": {}
}
```

Your eval run has now been queued, and it will execute asynchronously as it processes every row in your data set, generating responses for testing with the prompt and model we specified.

Analyze the results

To receive updates when a run succeeds, fails, or is canceled, create a webhook endpoint and subscribe to the `eval.run.succeeded`, `eval.run.failed`, and `eval.run.canceled` events. See the webhooks guide for more details.

Depending on the size of your dataset, the eval run may take some time to complete. You can view current status in the dashboard, but you can also fetch the current status of an eval run via API:

Retrieve eval run status

```
curl https://api.openai.com/v1/evals/YOUR_EVAL_ID/runs/YOUR_RUN_ID \\
    -H "Authorization: Bearer $OPENAI_API_KEY" \\
    -H "Content-Type: application/json"
```

```
import OpenAI from "openai";
const openai = new OpenAI();

const run = await openai.evals.runs.retrieve("YOUR_RUN_ID", {
    eval_id: "YOUR_EVAL_ID",
});
console.log(run);
```

```
from openai import OpenAI
client = OpenAI()

run = client.evals.runs.retrieve("YOUR_EVAL_ID", "YOUR_RUN_ID")
print(run)
```

You'll need the UUID of both your eval and eval run to fetch its status. When you do, you'll see eval run data that looks like this:


```
{
    "object": "eval.run",
    "id": "evalrun_67e44c73eb6481909f79a457749222c7",
    "eval_id": "eval_67e44c5becec81909704be0318146157",
    "report_url": "https://platform.openai.com/evaluation/evals/xxx",
    "status": "completed",
    "model": "gpt-4.1",
    "name": "Categorization text run",
    "created_at": 1743015028,
    "result_counts": {
        "total": 3,
        "errored": 0,
        "failed": 0,
        "passed": 3
    },
    "per_model_usage": [
        {
            "model_name": "gpt-4o-2024-08-06",
            "invocation_count": 3,
            "prompt_tokens": 166,
            "completion_tokens": 6,
            "total_tokens": 172,
```

```
            "cached_tokens": 0
        }
    ],
    "per_testing_criteria_results": [
        {
            "testing_criteria": "Match output to human label-40d67441-5000-
4754-ab8c-181c125803ce",
            "passed": 3,
            "failed": 0
        }
    ],
    "data_source": {
        "type": "responses",
        "source": {
            "type": "file_id",
            "id": "file-J7MoX9ToHXp2TutMEeYnwj"
        },
        "input_messages": {
            "type": "template",
            "template": [
                {
                    "type": "message",
                    "role": "developer",
                    "content": {
                        "type": "input_text",
                        "text": "You are an expert in categorizing IT support
tickets. Given the support ticket below, categorize the request into one of
Hardware, Software, or Other. Respond with only one of those words."
                    }
                },
                {
                    "type": "message",
                    "role": "user",
                    "content": {
                        "type": "input_text",
                        "text": "{{item.ticket_text}}"
                    }
                }
            ]
        },
        "model": "gpt-4.1",
        "sampling_params": null
    },
    "error": null,
    "metadata": {}
}
```

The API response contains granular information about test criteria results, API usage for generating model responses, and a `report_url` property that takes you to a page in the dashboard where you can explore the results visually.

In our simple test, the model reliably generated the content we wanted for a small test case sample. In reality, you will often have to run your eval with more criteria, different prompts, and different data sets. But the process above gives you all the tools you need to build robust evals for your LLM apps!

Next steps

Now you know how to create and run evals via API, and using the dashboard! Here are a few other resources that may be useful to you as you continue to improve your model results.

<a
href="https://cookbook.openai.com/examples/evaluation/use-cases/regression"
target="_blank"
rel="noreferrer"

<a
href="https://cookbook.openai.com/examples/evaluation/use-cases/bulk-experimentation"
target="_blank"
rel="noreferrer"

<a
href="https://cookbook.openai.com/examples/evaluation/use-cases/completion-monitoring"
target="_blank"
rel="noreferrer"

[

](https://developers.openai.com/api/docs/guides/fine-tuning)
[

](https://developers.openai.com/api/docs/guides/distillation)